

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

**Duration :60 Mins
Off Class
Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

I Ranjitha .R (192421285) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



1. Design a parser to recognize binary number strings.

Grammar

$S \rightarrow 0S$

$S \rightarrow 1S$

$S \rightarrow \epsilon$

Input

10110101

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the binary string.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

AIM:

To design and implement a parser that recognizes and validates binary number strings using the given grammar. The parser checks whether the input consists only of the binary digits 0 and 1, reports syntax errors for invalid characters, and displays whether the input string is Accepted or Rejected.

PROCEDURE:

1. Start the program.
2. Read the binary number from the user.
3. Store the input in a character array.
4. Traverse the input string one character at a time.
5. Check whether each character is either 0 or 1.
6. If an invalid character is found, display Rejected along with an appropriate syntax error message and terminate the program.
7. If all characters are valid binary digits, display Accepted.
8. End the program.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    int i;
```

```
    printf("Enter Binary Number: ");
```

```
    scanf("%s", str);
```

```
    for(i = 0; str[i] != '\0'; i++)
```

```

    {
        if(str[i] != '0' && str[i] != '1')
        {
            printf("Rejected\n");
            printf("Syntax Error: Invalid binary digit '%c'\n", str[i]);
            return 0;
        }
    }

    printf("Accepted\n");
    return 0;
}

```

OUTPUT:

The screenshot shows a C program in a code editor and its execution in a terminal. The code defines a function to check if a string is a valid binary number. It iterates through each character of the string, and if any character is not '0' or '1', it prints "Rejected" and "Syntax Error: Invalid binary digit" followed by the character, and returns 0. If all characters are '0' or '1', it prints "Accepted" and returns 0. The terminal output shows the program being run with the input "10110101", which is correctly identified as "Accepted".

```

1  #include <stdio.h>
2
3  int main()
4  {
5      char str[100];
6      int i;
7
8      printf("Enter Binary Number: ");
9      scanf("%s", str);
10
11     for(i = 0; str[i] != '\0'; i++)
12     {
13         if(str[i] != '0' && str[i] != '1')
14         {
15             printf("Rejected\n");
16             printf("Syntax Error: Invalid binary digit '%c'\n", str[i]);
17             return 0;
18         }
19     }
20
21     printf("Accepted\n");
22     return 0;
23 }

```

Enter Binary Number: 10110101
Accepted
...Program finished with exit code 0
Press ENTER to exit console.

RESULT:

Thus, the parser was successfully designed and implemented to recognize binary number strings. The program correctly validated the input, detected syntax errors for invalid characters, and displayed the appropriate Accepted or Rejected message.

- Design a parser to recognize octal number literals.

Grammar

$O \rightarrow \text{Digit } O$

$O \rightarrow \text{Digit}$

$\text{Digit} \rightarrow 0|1|2|3|4|5|6|7$

Input

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the octal number.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

AIM:

To design and implement a parser that recognizes and validates octal number literals using the given grammar. The parser verifies that the input contains only valid octal digits ranging from 0 to 7, identifies invalid symbols, and displays the appropriate acceptance or rejection message.

PROCEDURE:

1. Start the program.
2. Read the octal number from the user.
3. Store the input string in a character array.
4. Scan each character of the input one by one.
5. Verify that every digit lies between 0 and 7.
6. If any digit outside the valid octal range is found, display Rejected and print a syntax error message.
7. If all digits are valid, display Accepted.
8. Terminate the program.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    int i;
```

```
    printf("Enter Octal Number: ");
```

```
    scanf("%s", str);
```

```
    for(i = 0; str[i] != '\0'; i++)
```

```
    {
```

```
        if(str[i] < '0' || str[i] > '7')
```

```
        {
```

```
            printf("Rejected\n");
```

```
            printf("Syntax Error: Invalid octal digit '%c'\n", str[i]);
```

```
            return 0;
```

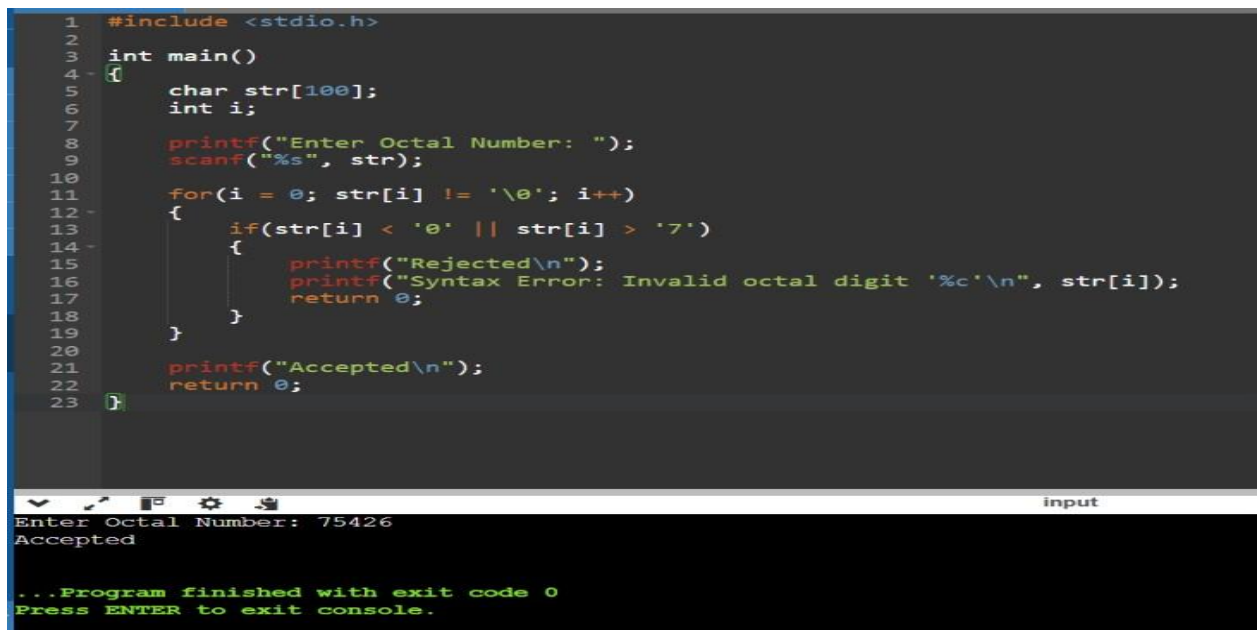
```
        }
```

```
    }
```

```
    printf("Accepted\n");
```

```
    return 0;
}
```

OUTPUT:



```
1  #include <stdio.h>
2
3  int main()
4  {
5      char str[100];
6      int i;
7
8      printf("Enter Octal Number: ");
9      scanf("%s", str);
10
11     for(i = 0; str[i] != '\0'; i++)
12     {
13         if(str[i] < '0' || str[i] > '7')
14         {
15             printf("Rejected\n");
16             printf("Syntax Error: Invalid octal digit '%c'\n", str[i]);
17             return 0;
18         }
19     }
20
21     printf("Accepted\n");
22     return 0;
23 }
```

Enter Octal Number: 75426
Accepted

...Program finished with exit code 0
Press ENTER to exit console.

RESULT:

Thus, the parser was successfully implemented to recognize octal number literals. It correctly validated valid octal numbers, detected invalid digits, and displayed the correct acceptance or rejection message.

3. Design a parser to recognize hexadecimal number literals.

Grammar

$H \rightarrow \text{HexDigit } H$

$H \rightarrow \text{HexDigit}$

$\text{HexDigit} \rightarrow 0|1|2|3|4|5|6|7|8|9|A|B|C|D|E|F$

Input

2AF9

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the hexadecimal number.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

AIM:

To design and implement a parser that recognizes and validates hexadecimal number literals using the given grammar. The parser verifies whether the input contains only valid hexadecimal symbols consisting of digits 0–9 and alphabetic characters A–F (or a–f) and reports syntax errors for invalid characters.

PROCEDURE:

1. Start the program.
2. Read the hexadecimal number from the user.
3. Store the input string.
4. Scan every character in the string.
5. Check whether each character is a valid hexadecimal digit (0–9, A–F, or a–f).
6. If an invalid character is encountered, display Rejected and print an appropriate syntax error message.
7. If all characters satisfy the hexadecimal format, display Accepted.
8. End the program.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    int i;
```

```
    printf("Enter Hexadecimal Number: ");
```

```
    scanf("%s", str);
```

```
    for(i = 0; str[i] != '\0'; i++)
```

```
    {
```

```
        if(!((str[i] >= '0' && str[i] <= '9') ||
```

```
            (str[i] >= 'A' && str[i] <= 'F') ||
```

```
            (str[i] >= 'a' && str[i] <= 'f')))
```

```
        {
```

```
            printf("Rejected\n");
```

```
            printf("Syntax Error: Invalid hexadecimal digit '%c'\n", str[i]);
```

```
            return 0;
```

```
        }
```

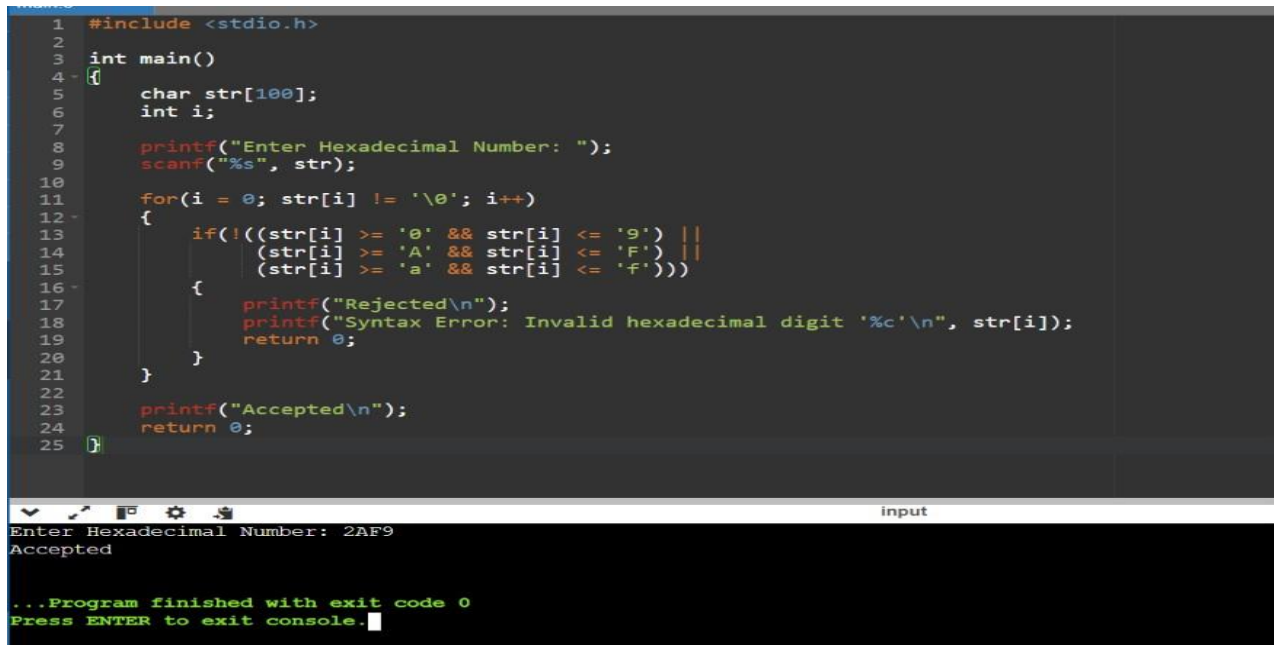
```
    }
```

```
    printf("Accepted\n");
```

```
    return 0;
```

```
}
```

OUTPUT:



```
1 #include <stdio.h>
2
3 int main()
4 {
5     char str[100];
6     int i;
7
8     printf("Enter Hexadecimal Number: ");
9     scanf("%s", str);
10
11     for(i = 0; str[i] != '\0'; i++)
12     {
13         if(!((str[i] >= '0' && str[i] <= '9') ||
14              (str[i] >= 'A' && str[i] <= 'F') ||
15              (str[i] >= 'a' && str[i] <= 'f'))))
16         {
17             printf("Rejected\n");
18             printf("Syntax Error: Invalid hexadecimal digit '%c'\n", str[i]);
19             return 0;
20         }
21     }
22
23     printf("Accepted\n");
24     return 0;
25 }
```

input

Enter Hexadecimal Number: 2AF9
Accepted

...Program finished with exit code 0
Press ENTER to exit console.

RESULT:

Thus, the parser was successfully designed and implemented to recognize hexadecimal number literals. The program accurately validated valid hexadecimal numbers, detected invalid symbols, and displayed the appropriate result.

4. Design a parser to recognize floating-point constants.

Grammar

$F \rightarrow \text{num} . \text{num}$

$F \rightarrow \text{num} . \text{num} \text{Exp}$

$\text{Exp} \rightarrow E + \text{num}$

$\text{Exp} \rightarrow E - \text{num}$

$\text{Exp} \rightarrow \epsilon$

Input

12.56E+03

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the floating-point constant.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

AIM:

To design and implement a parser that recognizes and validates floating-point constants according to the given grammar. The parser checks whether the input follows the correct floating-point format with an optional exponent part and reports syntax errors whenever the input does not match the required pattern.

PROCEDURE:

1. Start the program.
2. Read the floating-point constant from the user.
3. Store the input string.
4. Check whether the input contains a valid integer part, decimal point, and fractional part.
5. Verify the optional exponent part, if present, using E+ or E- followed by digits.
6. If the input does not satisfy the floating-point format, display Rejected and print an error message.
7. If the input matches the grammar, display Accepted.
8. Stop the program.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    printf("Enter Floating Point Constant: ");
```

```
    scanf("%s", str);
```

```
    if(sscanf(str, "%*d.%*dE%*[-+]*%*d") == 0)
```

```
    {
```

```
        printf("Accepted\n");
```

```
    }
```

```
    else if(sscanf(str, "%*d.%*d") == 0)
```

```
    {
```

```
        printf("Accepted\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("Rejected\n");
```

```
        printf("Syntax Error: Invalid floating-point constant\n");
```

```
    }
```

```
    return 0;
```

```
}
```

OUTPUT:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      char str[100];
6
7      printf("Enter Floating Point Constant: ");
8      scanf("%s", str);
9
10     if(sscanf(str, "%*d.%*dE%[-+]*%d") == 0)
11     {
12         printf("Accepted\n");
13     }
14     else if(sscanf(str, "%*d.%*d") == 0)
15     {
16         printf("Accepted\n");
17     }
18     else
19     {
20         printf("Rejected\n");
21         printf("Syntax Error: Invalid floating-point constant\n");
22     }
23
24     return 0;
25 }
```

input

Enter Floating Point Constant: 12.56E+03
Accepted

...Program finished with exit code 0
Press ENTER to exit console.

RESULT:

Thus, the parser was successfully implemented to recognize floating-point constants. The program correctly validated valid floating-point numbers with or without exponent notation, detected syntax errors, and displayed the appropriate acceptance or rejection message

5. Design a parser to recognize identifier lists.

Grammar

List $\rightarrow$ id

List $\rightarrow$ id , List

Input

a,b,c,d,e

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the identifier list.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

AIM:

To design and implement a parser that recognizes and validates a comma-separated list of identifiers according to the given grammar. The parser verifies that each identifier begins with a letter or underscore, contains only valid identifier characters, and is properly separated by commas.

PROCEDURE:

1. Start the program.
2. Read the identifier list from the user.

3. Store the input string.
4. Check that every identifier begins with a letter or underscore.
5. Verify that the remaining characters of each identifier contain only letters, digits, or underscores.
6. Ensure that identifiers are separated only by commas without any invalid symbols.
7. If any syntax error is found, display Rejected with an appropriate error message.
8. If the entire identifier list is valid, display Accepted.
9. End the program.

PROGRAM:

```
#include <stdio.h>
#include <ctype.h>
```

```
int main()
{
    char str[100];
    int i = 0;

    printf("Enter Identifier List: ");
    scanf("%s", str);

    while(str[i] != '\0')
    {
        if(!(isalpha(str[i]) || str[i] == '_'))
        {
            printf("Rejected\n");
            printf("Syntax Error: Identifier should start with a letter or '_'\n");
            return 0;
        }

        i++;

        while(isalnum(str[i]) || str[i] == '_')
            i++;

        if(str[i] == ',')
        {
            i++;

            if(str[i] == '\0')
            {
                printf("Rejected\n");
                printf("Syntax Error: Identifier expected after ','\n");
                return 0;
            }
        }
        else if(str[i] == '\0')
            return 0;
    }
}
```

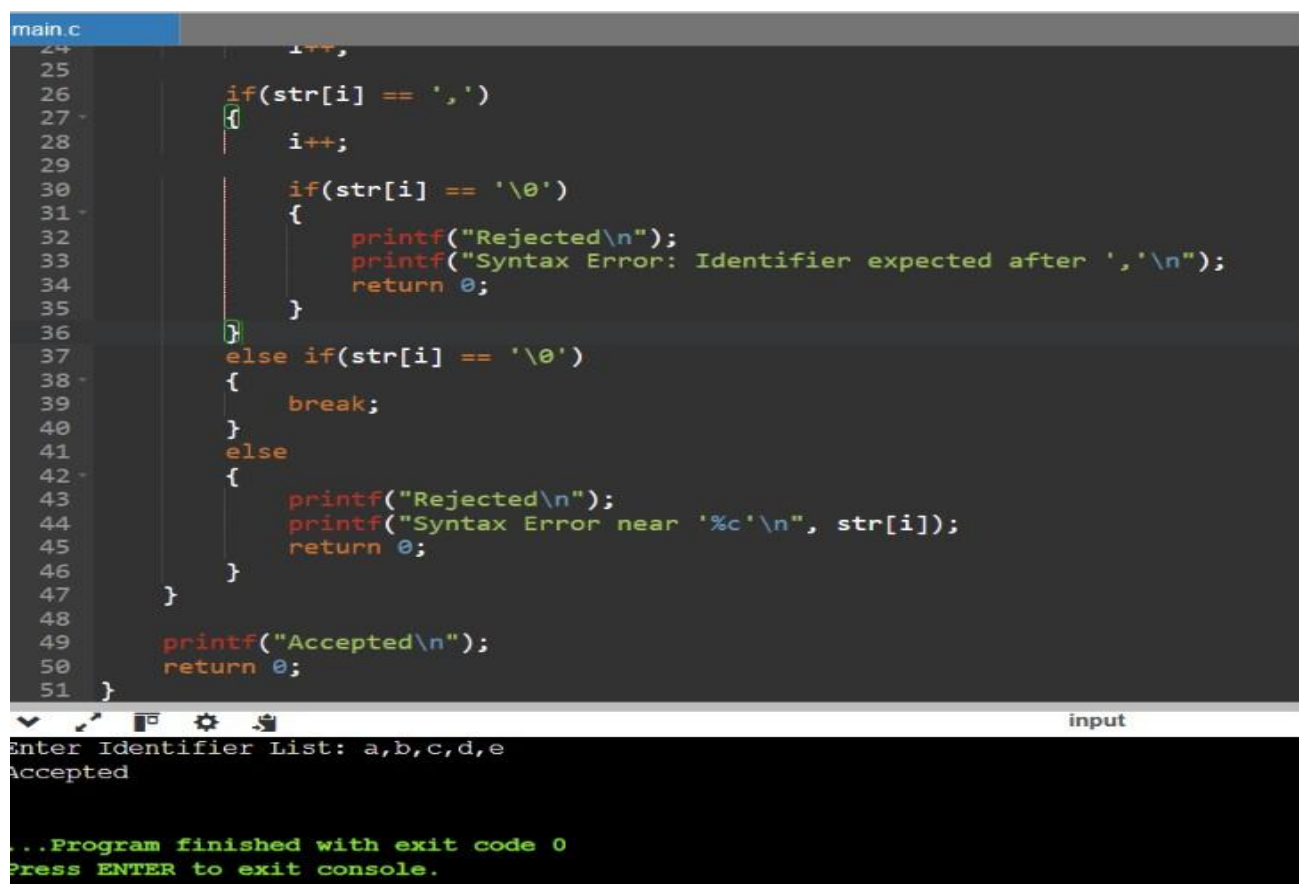
```

    {
        break;
    }
    else
    {
        printf("Rejected\n");
        printf("Syntax Error near '%c'\n", str[i]);
        return 0;
    }
}

printf("Accepted\n");
return 0;
}

```

OUTPUT:



The screenshot shows a code editor with a file named `main.c` containing the following C code:

```

24     i++;
25
26     if(str[i] == ',')
27     {
28         i++;
29
30         if(str[i] == '\0')
31         {
32             printf("Rejected\n");
33             printf("Syntax Error: Identifier expected after ','\n");
34             return 0;
35         }
36     }
37     else if(str[i] == '\0')
38     {
39         break;
40     }
41     else
42     {
43         printf("Rejected\n");
44         printf("Syntax Error near '%c'\n", str[i]);
45         return 0;
46     }
47 }
48
49 printf("Accepted\n");
50 return 0;
51 }

```

Below the code editor, the console output is shown:

```

input
Enter Identifier List: a,b,c,d,e
Accepted
...Program finished with exit code 0
Press ENTER to exit console.

```

RESULT:

Thus, the parser was successfully designed and implemented to recognize identifier lists. The program correctly validated the syntax of comma-separated identifiers, detected invalid identifier formats and separator errors, and displayed the appropriate Accepted or Rejected message. These expanded sections are suitable for writing in a college compiler design lab record.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	30	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	10	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation